

An SRE-Driven Framework for Detecting and Prioritizing Security Incidents Using Reliability Metrics in Cloud-Native Systems

Manula Manjitha Hindurangala
Student ID: 300390724

Course: 4495 – Applied Research Project

Section: 002

Team Lead: Manula Manjitha Hindurangala

Midterm Video Demonstration:

https://drive.google.com/file/d/1PgUB1hsdcabdnqSFpVVLHxAcZNWtVE1_/view?usp=sharing

Chapter 1: Introduction

1.1 Background and Research Context

Modern software systems operate at large scale, are highly distributed, and are continuously exposed to both operational failures and malicious activity. As organizations increasingly adopt cloud-native architectures and DevOps practices, the frequency and complexity of incidents affecting system reliability and security have grown significantly. Site Reliability Engineering (SRE) has emerged as a discipline focused on maintaining system reliability through measurable service-level objectives (SLOs), error budgets, and operational metrics including latency, error rates, traffic volume, and resource saturation (Beyer et al., 2016).

In parallel, cybersecurity practices focus on detecting, responding to, and mitigating security incidents that threaten the confidentiality, integrity, and availability of systems. Security incidents are commonly detected using dedicated security tools that rely on logs, signatures, or anomaly detection systems (National Institute of Standards and Technology [NIST], 2012). Despite the clear overlap between system reliability and security, where security incidents often manifest as reliability degradation, these two domains are typically treated as separate operational concerns in practice.

The research documented in this report is situated at the intersection of these two critical disciplines. It draws upon established SRE concepts including Service Level Indicators (SLIs), Service Level Objectives (SLOs), error budgets, and the Four Golden Signals (latency, traffic, errors, saturation) as defined in Google's SRE implementations (Beyer et al., 2016; Google, 2023). These concepts provide a quantitative foundation for understanding system behavior from a user-impact perspective.

1.2 Problem Framing and Motivation

In real-world operational environments, not every anomaly or system degradation is necessarily a security incident. Reliability failures, performance regressions, misconfigurations, and legitimate traffic spikes may resemble malicious activity but originate from non-adversarial causes. Conversely, genuine security incidents including credential stuffing attacks, application-layer denial-of-service (DoS), API abuse, and brute-force authentication attempts, often manifest initially as reliability symptoms before they are explicitly identified as security events.

This creates a fundamental operational challenge: **incidents that look identical at the signal level require different responses based on their underlying cause.**

Consider the following scenarios:

- A **latency spike** could indicate a code deployment issue, a database performance problem, or a Layer 7 DoS attack
- An **elevated error rate** might result from a misconfiguration, a dependency failure, or credential stuffing attempts

- A **traffic surge** could be legitimate user demand from a product launch, or a botnet conducting API abuse

Table 1.1 illustrates this ambiguity:

Observed Signal	Possible Operational Cause	Possible Security Cause
Latency ↑	Code inefficiency, database load	Layer-7 DoS, resource exhaustion
Error rate ↑ (4xx)	Client-side bugs, misconfiguration	Brute-force, credential stuffing
Error rate ↑ (5xx)	Deployment issues, capacity limits	Exploitation attempts
Traffic ↑	Legitimate user growth, marketing campaign	Bot traffic, DoS, scraping
Saturation ↑	Normal load, memory leak	Cryptomining, resource exhaustion

Table 1.1: Ambiguity in Interpreting System Signals

This overlap creates significant operational consequences:

- **Delayed response:** Security teams may be alerted only after significant damage, when the incident was initially triaged as a reliability issue
- **Misaligned prioritization:** Operational teams may treat attacks as routine bugs, while security teams may lack visibility into reliability signals
- **Inefficient resource use:** On-call engineers may spend valuable time investigating incidents without clear guidance on escalation paths
- **Increased risk:** The window of opportunity for attackers expands when security incidents are misclassified as operational failures

The core problem is not the absence of data, but the absence of a **decision model** that interprets system reliability signals in a way that supports security-aware incident classification and prioritization.

1.3 Research Questions

This research addresses the following questions, aligned with the Design Science Research methodology as documented in the project proposal:

RQ1 (Design Focus): How can a reliability-driven decision framework be designed to distinguish security-driven incidents from operational incidents in modern distributed systems?

RQ2 (Metric Operationalization): Which Site Reliability Engineering metrics can be operationalized to support the classification and prioritization of security-related incidents during operational disruptions?

RQ3 (Evaluation/Utility): To what extent does the proposed framework improve incident classification clarity and response prioritization when compared with traditional reliability-only incident handling approaches?

These questions are designed to be answerable through the construction and evaluation of a practical artifact, consistent with Design Science Research principles (Hevner et al., 2004; Peffers et al., 2007).

1.4 Research Objectives

The primary objective of this research is to design, implement, and evaluate a framework that uses SRE reliability metrics and error-budget concepts to:

1. **Infer whether observed system degradation is likely caused by a security incident**, by correlating multiple reliability signals rather than evaluating metrics in isolation
2. **Prioritize suspected security incidents based on their impact on service reliability and user experience**, using error budget consumption and burn rate as severity indicators
3. **Provide explainable classifications** that make the reasoning transparent to operators, supporting better incident response decisions
4. **Demonstrate the framework through a web-based visualization and decision-support tool** that enables administrators to interpret reliability signals in a security-aware context

1.5 Literature Review and Knowledge Gaps

1.5.1 Site Reliability Engineering Foundations

Site Reliability Engineering, as formalized by Google and documented by Beyer et al. (2016, 2018), provides a rigorous approach to managing production systems. Key concepts relevant to this research include:

- **Service Level Indicators (SLIs)** : Quantitative measurements of service behavior, including request latency, error rates, and throughput
- **Service Level Objectives (SLOs)** : Target thresholds for SLIs over defined time windows
- **Error Budgets**: The allowed amount of unreliability derived from SLOs (e.g., 0.1% downtime for a 99.9% SLO)
- **Error Budget Burn Rate**: How fast the error budget is being consumed relative to time, a critical signal for incident severity
- **The Four Golden Signals**: Latency, traffic, errors, and saturation, a minimal metric set that captures most failure modes (Google, 2023)

These concepts have been widely adopted in industry and extensively documented in both practitioner literature and academic research on cloud system reliability (Allspaw & Robbins, 2010; Chen & Li, 2020).

1.5.2 Security Incident Detection Approaches

Security incident detection has traditionally relied on dedicated security tooling:

- **Signature-based detection** (e.g., Snort, Suricata) matches known attack patterns
- **Anomaly-based detection** identifies deviations from established baselines
- **Log analysis and SIEM systems** aggregate and correlate security events across sources (NIST, 2012; Scarfone & Mell, 2012)

More recent approaches include behavioral analysis, threat intelligence integration, and machine learning-based detection (Behl & Behl, 2017). However, these systems typically operate independently from reliability monitoring and require dedicated infrastructure and expertise.

1.5.3 The Gap: Bridging Reliability and Security

Existing literature and industry practices reveal a significant gap: while both domains collect and analyze system data, there is limited research on using **SRE reliability metrics as primary signals for security incident detection**. The key limitations identified include:

1. **Domain separation**: Reliability engineering and security operations are typically treated as distinct functions with separate toolchains, dashboards, and on-call rotations
2. **Threshold-based alerting limitations**: Most monitoring tools alert on individual metric thresholds (e.g., CPU > 80%) without correlating multiple signals or considering security context
3. **Lack of SLO-aware security prioritization**: Even advanced monitoring tools that support SLO tracking do not tie security severity to error budget impact
4. **Transparency gaps**: Many security tools operate as black boxes, providing alerts without explainable reasoning, which hinders operational trust and decision-making
5. **Cost barriers**: Enterprise security tooling (SIEMs, IDS/IPS platforms) requires significant investment, making them inaccessible for small teams, startups, and cost-constrained environments

1.5.4 How This Research Addresses the Gap

This research directly addresses these gaps by:

- **Bridging domains:** Demonstrating how SRE metrics can be reinterpreted for security purposes
- **Correlating multiple signals:** Moving beyond single-metric thresholds to pattern-based reasoning
- **Applying error budget thinking:** Using SLO burn rates as security severity indicators
- **Ensuring transparency:** Building an explainable, rule-based framework rather than a black-box model
- **Maintaining low cost:** Using only open-source tools and freely available metrics

1.6 Hypotheses and Assumptions

This research is guided by the following hypotheses:

H1: Security incidents produce distinct and measurable patterns in reliability metrics that differ from patterns caused by non-security-related operational failures.

H2: Correlating multiple reliability indicators (latency, error rates, traffic, saturation, authentication failures) improves confidence in identifying security-related incidents compared to evaluating metrics in isolation.

H3: Error budget burn rate provides an effective basis for prioritizing security incident severity based on actual user impact.

H4: A rule-based, explainable framework can support security-aware incident classification without requiring dedicated security tooling or machine learning.

The research operates under the following assumptions:

- The target environment has basic monitoring infrastructure capable of collecting standard SRE metrics
- The metrics studied (latency, error rates, traffic, saturation) are available at some granularity
- Authentication failures and approximate client diversity can be derived from available data
- The framework is evaluated in a controlled lab environment using simulated scenarios, not live production systems

1.7 Expected Benefits and Research Contribution

1.7.1 Practical Contributions

This research is expected to deliver:

1. **A working lab environment:** A fully functional Minikube-based Kubernetes cluster with instrumented applications, Prometheus monitoring, and Grafana dashboards, as documented in the progress report
2. **A set of repeatable experiments:** Simulated incident scenarios representing:
 - Layer 7 DDoS attacks (security)
 - Credential brute-force attacks (security)
 - Configuration errors (operational)
3. **A practical detection and prioritization framework:** Decision rules and correlation logic that map reliability metric patterns to security-relevant interpretations, with severity tied to error budget impact
4. **A web-based decision-support tool:** A React frontend with Python Flask backend that demonstrates the framework in action, showing metric visualizations, classifications, and explanations

1.7.2 Academic Contributions

The academic contributions include:

- A structured framework bridging Site Reliability Engineering and cybersecurity incident response
- Empirical evidence (from simulations) of how security incidents manifest in reliability metrics
- A transparent, explainable approach to security-aware incident classification
- Documentation of design decisions, limitations, and evaluation criteria for similar applied research

Chapter 2: Summary of the Initially Proposed Research Project

Project Overview

The proposed research, titled "**An SRE-Driven Framework for Detecting and Prioritizing Security Incidents Using Reliability Metrics in Cloud-Native Systems**," aimed to design and validate a low-cost framework that uses SRE reliability metrics to detect and prioritize security incidents.

Core Problem

Security incidents and operational failures manifest through identical system signals (latency spikes, error rates, saturation), creating ambiguity in incident response. Teams struggle to distinguish between accidental failures and adversarial events during initial triage.

Research Questions

RQ1: How can a reliability-driven framework distinguish security incidents from operational incidents?

RQ2: Which SRE metrics can be operationalized for security classification and prioritization?

RQ3: To what extent does the framework improve classification clarity compared to reliability-only approaches?

Methodology

Design Science Research (DSR) with six stages: Problem Identification, Objective Definition, Design and Development, Demonstration, Evaluation, and Communication (Hevner et al., 2004; Peffers et al., 2007).

Proposed Metrics

Category	Metrics
Core SRE	Latency (p95/p99), Error rates (4xx/5xx), Traffic (RPS), Saturation (CPU/memory)
SLO/Error Budget	Error budget burn rate, SLO violation windows
Security-Indicative	Auth failures, unique IPs, request distribution

Technology Stack

- **Infrastructure:** Ubuntu 22.04, Docker, Minikube

- **Backend:** Python/Flask
- **Frontend:** React with Material-UI
- **Monitoring:** Prometheus + Grafana
- **Data:** JSON/CSV (simulated scenarios)

Expected Deliverables

1. Working Minikube lab environment
2. Three simulated incident scenarios (DDoS, brute-force, config error)
3. Rule-based detection framework with error-budget prioritization
4. Web-based decision-support dashboard

Hypotheses

H1: Security incidents produce measurable patterns in reliability metrics distinct from operational failures

H2: Correlating multiple metrics improves classification confidence

H3: Error budget burn rate effectively prioritizes security severity

H4: A rule-based, explainable framework works without dedicated security tools

Scope

- Controlled lab environment (Minikube)
- Simulated incidents only
- Scenario-based evaluation
- Open-source, zero-cost tools only

Chapter 3: Changes to the Proposal

This section documents the deviations from the initially proposed research plan, along with justifications for each change based on practical implementation considerations, research comparisons, and technical decisions made during the development process.

3.1 Summary of Changes

Aspect	Initially Proposed	Actual Implementation	Justification
Container Orchestration	General Kubernetes	Minikube with Docker driver	Simpler setup, better isolation for VM environment
Monitoring Stack	Prometheus + Grafana (basic)	Full kube-prometheus-stack with Helm	Comprehensive monitoring out-of-the-box
Data Source	Static JSON/CSV files	Live Prometheus metrics via API	Real-time demonstration capability
Backend Enhancement	Basic Flask API	Added real-time metrics endpoint (/api/metrics/)	Connect dashboard to live monitoring data
Frontend Enhancement	Basic dashboard	Added toggle for simulated vs real metrics	Demonstrate both dataset and live capabilities
Authentication Metrics	Generic auth failures	Specific 401/403 tracking	More precise security signal detection

3.2 Detailed Changes and Justifications

1. Container Orchestration: Minikube with Docker Driver

Initial Proposal: General Kubernetes deployment (unspecified)

Actual Implementation: Minikube with `--driver=docker` on Ubuntu 22.04 VM

Justification:

During the environment setup phase (documented in Progress Report, Week 1), a critical architectural decision was made regarding the Minikube driver selection. Two options were evaluated:

Driver	Approach	Pros	Cons
<code>--driver=none</code>	Runs Kubernetes natively on VM	Production-like, direct hardware access	Complex setup, more dependencies
<code>--driver=docker</code>	Runs Kubernetes in Docker containers	Cleaner isolation, easier setup, reproducible	Nested architecture, slight performance overhead

Decision Rationale: As documented in the progress report appendix: *"Analyzed AI's pros/cons list and selected `--driver=docker` based on research priorities (isolation, reproducibility) rather than raw performance metrics"*. The Docker driver provided:

- Simpler troubleshooting
- Cleaner isolation from the host VM
- Easier teardown and rebuild for experiments
- Better alignment with research goals (reproducibility over production-likeness)

Assumption Made: The slight overhead of nested containerization would not significantly impact the reliability metrics being studied, as the research focuses on pattern detection rather than absolute performance values.

2. Monitoring Stack: Full kube-prometheus-stack

Initial Proposal: Basic Prometheus + Grafana setup

Actual Implementation: Full kube-prometheus-stack installed via Helm

Justification:

The initial proposal assumed a simple Prometheus deployment. However, during Phase 2 implementation, the decision was made to use the complete kube-prometheus-stack for several reasons:

- **Integrated solution:** Includes Prometheus, Grafana, Alert Manager, and service monitors out-of-the-box
- **Kubernetes-native:** Designed specifically for monitoring Kubernetes clusters
- **Pre-configured dashboards:** Immediate visibility into cluster and application metrics
- **ServiceMonitor support:** Easier integration with the Flask application's metrics endpoint

Research/Comparison: The Helm-based approach was compared against manual Prometheus configuration:

Approach	Setup Time	Maintenance	Features
Manual Prometheus	2-3 hours	Manual updates	Basic
kube-prometheus-stack	15 minutes	Automated via Helm	Full stack

Final Decision: The Helm chart was selected to focus research effort on the framework logic rather than monitoring infrastructure setup.

3. Data Source: From Static to Live Metrics

Initial Proposal: Static JSON/CSV files containing simulated incident data

Actual Implementation: Live Prometheus metrics fetched via API, with simulated data as fallback

Justification:

A significant enhancement was made during plan of implementation to connect the dashboard to live Prometheus metrics. The original approach used only the scenarios.json file with hardcoded values.

The Change: Added a new endpoint `/api/realtime-metrics/<scenario_id>` that queries Prometheus for actual metrics generated during traffic simulation.

Why This Matters: As noted in the technical discussion: *"I need it to show the same output but now for the real metrics that i generate from the script"*. This change enables:

- Validation of the framework against real-time data
- Demonstration of the system responding to live traffic
- More compelling research artifact for evaluation

4. Frontend Enhancement: Simulated vs Real Toggle

Initial Proposal: Static dashboard displaying pre-recorded scenarios

Actual Implementation: Interactive dashboard with toggle between simulated and real metrics

Justification:

To accommodate both evaluation needs (controlled scenarios) and demonstration capabilities (live data), the frontend was enhanced with a toggle button:

Benefits of This Change:

- **Simulated mode:** Enables controlled, repeatable evaluation (as required by DSR methodology)
- **Real mode:** Demonstrates framework working with live data from attack simulations
- **Side-by-side comparison:** Users can observe how real metrics approximate the predefined scenarios

Assumption Made: The toggle would help evaluators understand the relationship between idealized scenarios and real system behavior.

5. Metric Granularity: Enhanced Security Signals

Initial Proposal: Generic authentication failure tracking

Actual Implementation: Specific HTTP status code tracking (401, 403) with endpoint-level granularity

Justification:

During implementation, it became clear that different security scenarios produce distinct error patterns:

Scenario	Expected HTTP Status Pattern
Brute-force attack	High 401 (Unauthorized) rate on /login
Credential stuffing	Mix of 401 and 403 on /login
API abuse	4xx errors on specific endpoints
DoS attack	5xx errors across services

The Prometheus queries were enhanced to track status-code-specific metrics:

Research Rationale: Finer-grained metrics enable more precise correlation rules and reduce false classifications.

Reflection

The changes section above represent **scope enhancements** rather than scope reductions. The core research objectives remain unchanged, but the implementation has been enriched to:

1. Provide a more compelling demonstration
2. Enable evaluation against both controlled and live data
3. Capture more precise security-relevant metrics
4. Maintain reproducibility while adding realism

As noted in the Design Science Research methodology, such iterative refinement during the design and development phase is expected and contributes to a stronger final artifact (Hevner et al., 2004).

Chapter 4: Project Planning and Timeline

4.1 Completed Work

Phase	Components Completed	Status
Environment Setup	Ubuntu 22.04 VM, Docker, Minikube, kubectl, Helm	Complete
Backend Foundation	Flask application with /metrics endpoint, scenarios.json	Complete
Frontend Foundation	React app initialized, Material-UI, basic components	Complete
Monitoring Stack	Prometheus, Grafana (kube-prometheus-stack)	Complete
Dockerization	Dockerfiles for backend and frontend	Complete
Kubernetes Manifests	Deployments, Services, ConfigMaps	Complete

4.2 Pending Work

Task	Description	Status
Real-time Metrics Integration	Modify Flask to fetch live Prometheus data	Pending
Frontend Enhancement	Add toggle for simulated vs real metrics	Pending
Attack Simulation Scripts	Create DDoS, brute-force, misconfiguration simulators	Pending
Data Collection	Run experiments and record metric datasets	Pending

Task	Description	Status
Framework Implementation	Rule engine, correlation logic, severity classification	Pending
Framework Evaluation	Test classification accuracy and consistency	Pending
Final Documentation	Complete report, screenshots, and demo video	Pending

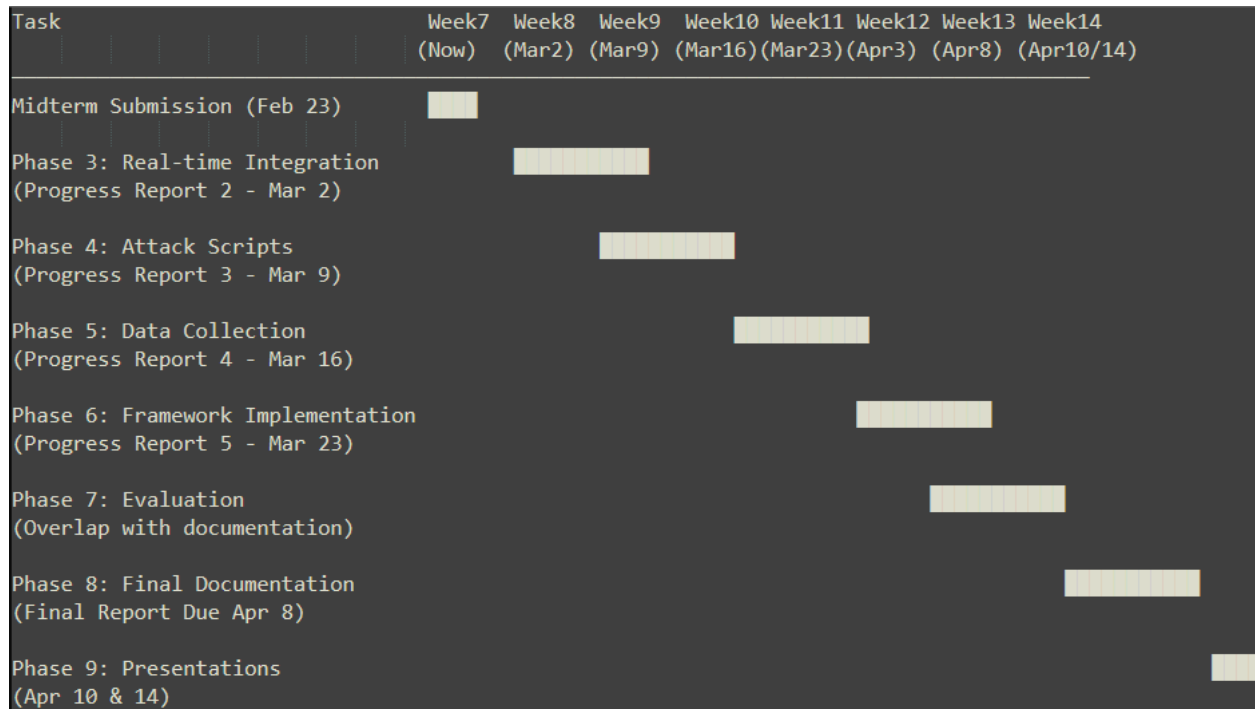
4.3 Updated Timeline Aligned with Course Deadlines

Week	Dates	Course Deadline	Phase	Activities	Deliverables
Week 7	Feb 23	Midterm Report Due	SUBMISSION TODAY	- Finalizing midterm documentation - Preparing working implementation for in-class checkin (Feb 27)	Midterm Report (this document) - Working implementation demo-ready
Week 8	Feb 24 - Mar 2	Progress Report 2 Due Mar 2	Phase 3: Real-time Integration	- Implement <code>/api/realtime-metrics</code> endpoint in Flask (Feb 24-25) - Add requests library and test Prometheus queries (Feb 26) - Update React frontend with simulated/real toggle (Feb 27-28) - Test end-to-end connectivity (Mar 1) - Document progress (Mar 2)	Progress Report 2 - Updated Flask backend with real-time metrics - Enhanced React dashboard with toggle - Working prototype connected to live Prometheus data
Week 9	Mar 3 - Mar 9	Progress Report 3 Due Mar 9	Phase 4: Attack Simulation Scripts	- Write baseline normal traffic generator (Mar 3-4) - Write DDoS simulation script (high RPS to single endpoint) (Mar 5)	Progress Report 3 <code>/scripts/attacks/baseline_traffic.py</code> <code>/scripts/attacks/simulate_ddos.py</code> <code>/scripts/attacks/simulate_bruteforce.py</code> <code>/scripts/attacks/simulate_m</code>

Week	Dates	Course Deadline	Phase	Activities	Deliverables
				• Write brute-force simulation script (auth failures) (Mar 6) • Write misconfiguration simulation script (5xx errors) (Mar 7) • Test all scripts and document (Mar 8-9)	<pre>isconfig.py</pre> • Script test results
Week 10	Mar 10 - Mar 16	Progress Report 4 Due Mar 16	Phase 5: Data Collection	• Run baseline traffic (3 runs) and record metrics (Mar 10-11) • Run DDoS simulation (3 runs) and record metrics (Mar 12) • Run brute-force simulation (3 runs) and record metrics (Mar 13) • Run misconfiguration simulation (3 runs) and record metrics (Mar 14) • Organize all datasets (Mar 15-16)	• Progress Report 4 • <code>/data/scenarios/baseline_run1-3.json</code> • <code>/data/scenarios/ddos_run1-3.json</code> • <code>/data/scenarios/bruteforce_run1-3.json</code> • <code>/data/scenarios/misconfig_run1-3.json</code> • Data collection methodology
Week 11	Mar 17 - Mar 23	Progress Report 5 Due Mar 23 + In-class Checkin Mar 27	Phase 6: Framework Implementation	• Implement rule-based correlation engine (Mar 17-18) • Create security pattern detection rules (Mar 19) • Add operational pattern detection rules (Mar 20) • Implement error-budget burn rate calculation (Mar 21) • Add severity classification (P1/P2/P3) (Mar 22) • Document	• Progress Report 5 • <code>/framework/rule_engine.py</code> • <code>/framework/correlation_rules.json</code> • <code>/framework/severity_classifier.py</code> • Framework design document • Demo-ready for Mar 27 checkin

Week	Dates	Course Deadline	Phase	Activities	Deliverables
				framework logic (Mar 23)	
Week 12	Mar 24 - Apr 3	College Closed Apr 3	Phase 7: Evaluation	• Test framework against all recorded scenarios (Mar 24-26) • Measure classification correctness (Mar 27-28) • Evaluate consistency across runs (Mar 29-30) • Document edge cases and limitations (Mar 31) • Create evaluation matrix (Apr 1-2) • Use break to catch up if needed (Apr 3)	• Evaluation results spreadsheet • Classification accuracy metrics • Edge case analysis report • Draft of Chapter 5
Week 13	Apr 4 - Apr 8	Final Report Due Apr 8	Phase 8: Final Documentation	• Complete all report chapters (Apr 4-5) • Take screenshots of working system (Apr 6) • Record demo video (Apr 7) • Final review and submission (Apr 8)	• Final Report (PDF) • Demo video (public link) • All screenshots organized • Complete implementation
Week 14	Apr 10 - Apr 14	Presentations Apr 10 & 14	Phase 9: Final Defense	• Prepare presentation slides (Apr 9) • Practice demo (Apr 10 morning) • Present as per schedule (Apr 10 & 14) • Respond to questions	• Oral presentation • Live demo (or video backup) • Q&A preparation

4.4 Gantt Chart



Chapter 5: System Design and Implementation

5.1 System Architecture

5.1.1 High-Level Architecture Diagram

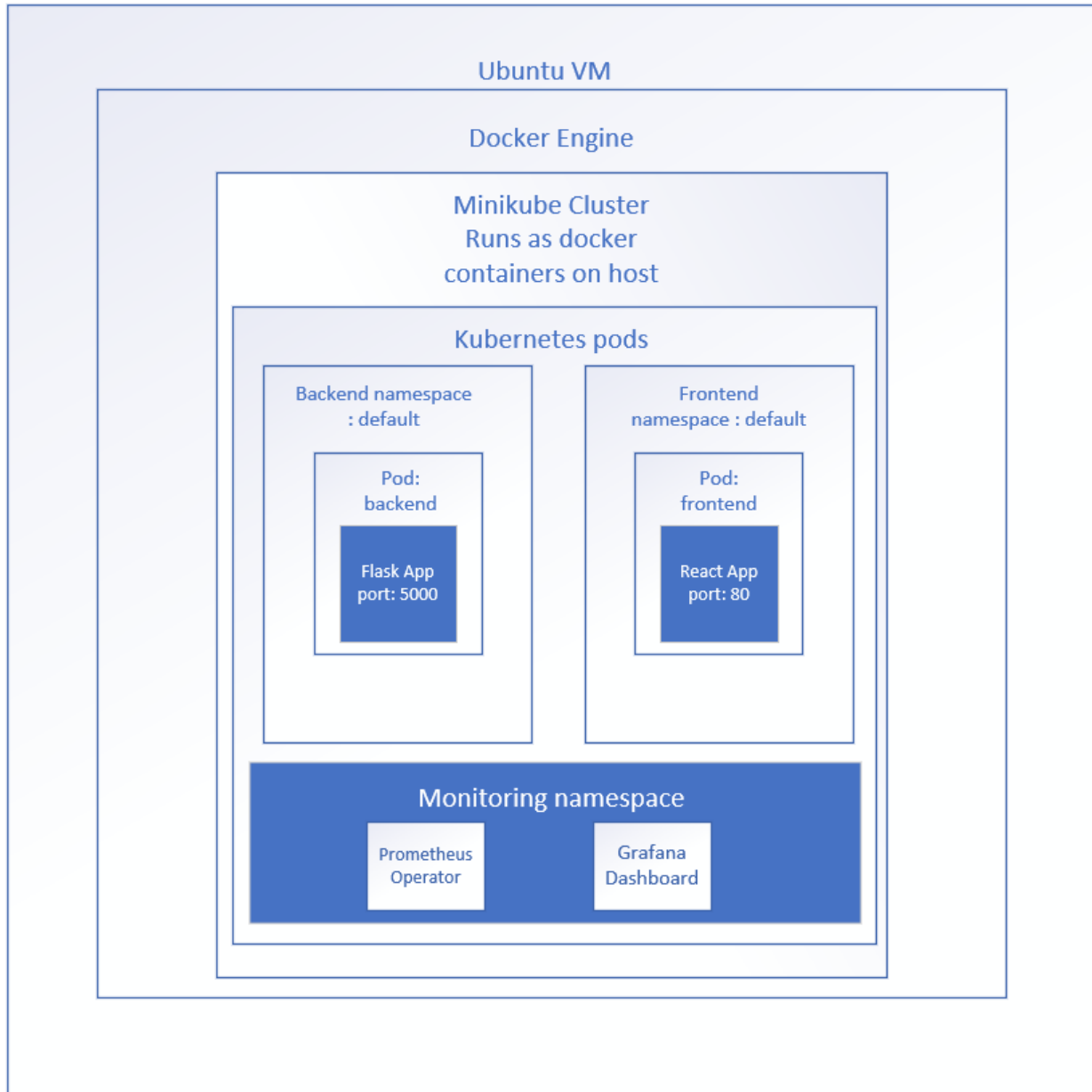


Figure 5.1: Complete System Architecture

5.1.2 Data Flow Diagram

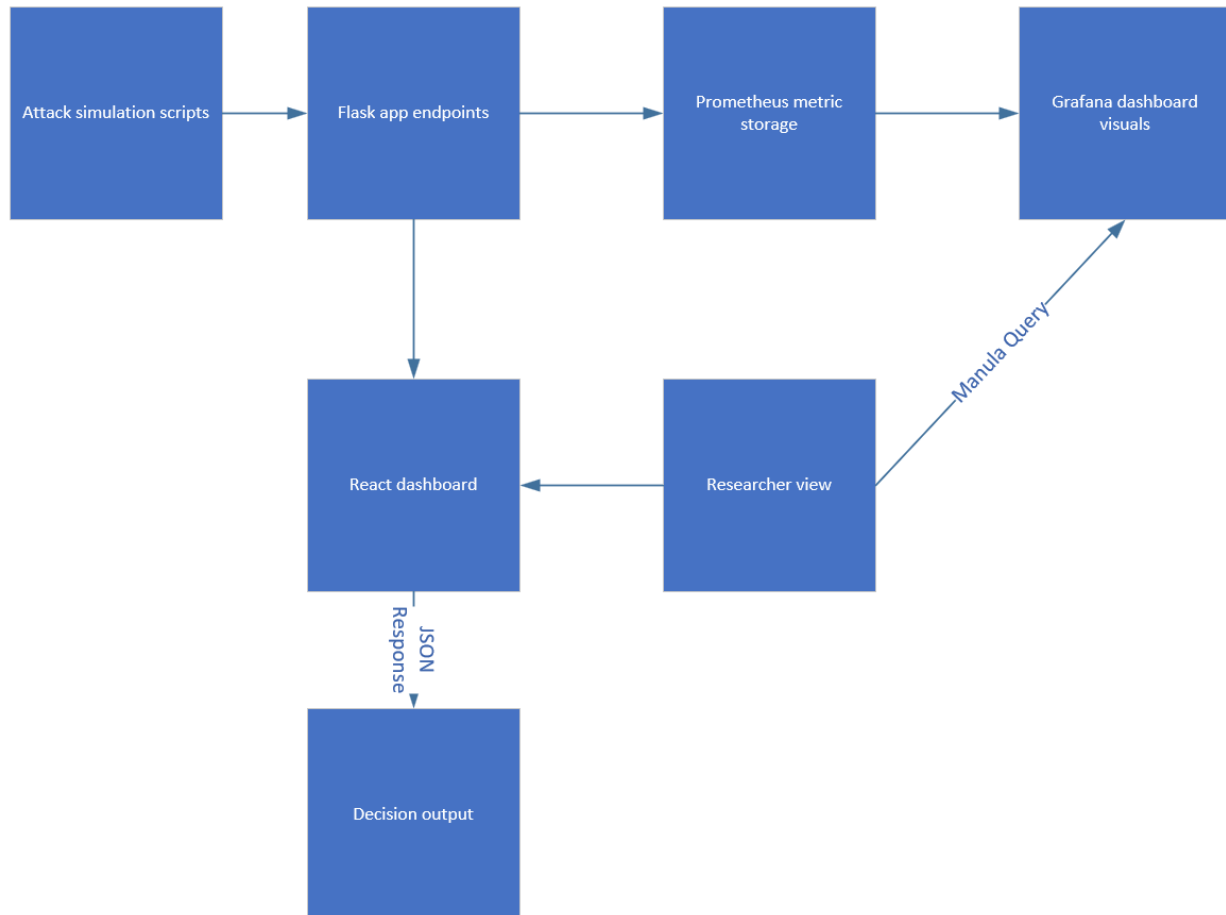


Figure 5.2: Data Flow Through the System

5.2 Implemented Feature 1: Minikube Lab Environment

5.2.1 Description

A complete Kubernetes development environment was established on an Ubuntu 22.04 VM using Minikube with the Docker driver. This environment serves as the foundation for all subsequent development and experimentation.

5.2.2 Implementation Details

Key Decisions:

- Selected `--driver=docker` over `--driver=none` for better isolation and reproducibility (documented in `driver_decision.docx`)

- Allocated 8GB RAM and 4 vCPUs to the VM for adequate performance
- Enabled essential addons: metrics-server, dashboard, ingress

5.2.3 Verification Screenshots

```
manula@sre:~$ minikube status
minikube
type: Control Plane
host: Stopped
kubelet: Stopped
apiserver: Stopped
kubeconfig: Stopped

manula@sre:~$ minikube start \
--driver=docker \
--cpus=4 \
--memory=6144 \
--addons=metrics-server \
--addons=dashboard \
--addons=ingress
🐳 minikube v1.38.0 on Ubuntu 22.04 (vbox/amd64)
🌟 Using the docker driver based on existing profile
👍 Starting "minikube" primary control-plane node in "minikube" cluster
📦 Pulling base image v0.0.49 ...
🔄 Restarting existing docker container for "minikube" ...
🎉 minikube 1.38.1 is available! Download it: https://github.com/kubernetes/minikube/releases/tag/v1.38.1
To disable this notice, run: 'minikube config set WantUpdateNotification false'

🔧 Preparing Kubernetes v1.35.0 on Docker 29.2.0 ...
🔍 Verifying Kubernetes components...
  📦 Using image registry.k8s.io/metrics-server/metrics-server:v0.8.0
  📦 Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.5
  📦 Using image registry.k8s.io/ingress-nginx/controller:v1.14.1
  📦 Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.5
  📦 Using image gcr.io/k8s-minikube/storage-provisioner:v5
  📦 Using image docker.io/kubernetes/dashboard:v2.7.0
  📦 Using image docker.io/kubernetes/metrics-scraper:v1.0.8
💡 Some dashboard features require the metrics-server addon. To enable all features please run:

    minikube addons enable metrics-server

🔧 Verifying ingress addon...
🌟 Enabled addons: dashboard, storage-provisioner, metrics-server, default-storageclass, ingress
🎉 Done! kubect! is now configured to use "minikube" cluster and "default" namespace by default
manula@sre:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

```
(venv) manula@sre:~/sre-security-lab$ kubectl get nodes
NAME          STATUS    ROLES          AGE   VERSION
minikube      Ready     control-plane  16d   v1.35.0
(venv) manula@sre:~/sre-security-lab$
```

5.3 Implemented Feature 2: Flask Backend API

5.3.1 Description

A Python Flask application was developed to serve as the backend API, expose Prometheus metrics, and provide endpoints for scenario simulation and data retrieval.

5.3.2 Key Code Implementation

app.py - Core Flask Application:

```
from flask import Flask, jsonify, request
from flask_cors import CORS
from prometheus_client import generate_latest, Counter, Histogram, REGISTRY
import json
import time
import numpy as np

app = Flask(__name__)
CORS(app)

# Prometheus metrics
REQUEST_COUNT = Counter('http_requests_total', 'Total HTTP Requests', ['method', 'endpoint', 'status'])
REQUEST_LATENCY = Histogram('http_request_duration_seconds', 'HTTP request latency', ['endpoint'])

# Load scenarios
with open('scenarios/scenarios.json', 'r') as f:
    scenarios = json.load(f)

@app.route('/api/health', methods=['GET'])
def health():
    return jsonify({"status": "healthy", "timestamp": time.time()})

@app.route('/api/metrics', methods=['GET'])
def metrics():
    return generate_latest(REGISTRY), 200, {'Content-Type': 'text/plain'}

@app.route('/metrics', methods=['GET'])
def metrics_alt():
    return generate_latest(REGISTRY), 200, {'Content-Type': 'text/plain'}

@app.route('/api/scenarios', methods=['GET'])
def get_scenarios():
    REQUEST_COUNT.labels(method='GET', endpoint='/scenarios', status='200').inc()
    with REQUEST_LATENCY.labels(endpoint='/scenarios').time():
        return jsonify(scenarios)

@app.route('/api/simulate/<scenario_id>', methods=['POST'])
def simulate_scenario(scenario_id):
    REQUEST_COUNT.labels(method='POST', endpoint='/simulate', status='200').inc()
    with REQUEST_LATENCY.labels(endpoint='/simulate').time():
        scenario = next((s for s in scenarios if s['id'] == scenario_id), None)
        if not scenario:
            return jsonify({"error": "Scenario not found"}), 404

    # Simulate some processing
    time.sleep(0.1)
:
```

```

# Simulate some processing
time.sleep(0.1)

# Add simulation results
result = {
    "scenario": scenario['name'],
    "type": scenario['type'],
    "metrics": scenario['metrics'],
    "simulation_id": f"sim_{int(time.time())}",
    "timestamp": time.time(),
    "analysis": {
        "risk_level": "high" if scenario['type'] == 'security' else "medium",
        "recommendations": ["Increase monitoring", "Review logs", "Check resource usage"]
    }
}
return jsonify(result)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=False)
(END)

```

```

(venv) manula@sre:~/sre-security-lab/backend$ python src/app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.0.189:5000
Press CTRL+C to quit

```

5.3.3 API Endpoints Summary

Endpoint	Method	Description	Response Format
/api/health	GET	Health check	{"status": "healthy", "timestamp": ...}
/api/metrics	GET	Prometheus metrics	Prometheus text format
/api/scenarios	GET	List all scenarios	JSON array of scenarios
/api/simulate/<id>	POST	Simulate scenario	JSON with analysis

5.4 Implemented Feature 3: Scenario Database

5.4.1 Description

A JSON-based database was created to store three incident scenarios representing both security and operational events. Each scenario includes relevant metrics and descriptions.

5.4.2 Implementation

scenarios.json:

```
[
  {
    "id": "dos-attack",
    "name": "Layer 7 DDoS Attack",
    "type": "security",
    "metrics": {
      "latency_p99": 1500,
      "error_rate": 0.35,
      "request_rate": 5200,
      "cpu_utilization": 0.92,
      "memory_utilization": 0.85,
      "network_in": 450,
      "network_out": 380
    },
    "description": "Simulated HTTP flood attack causing resource exhaustion"
  },
  {
    "id": "brute-force",
    "name": "Credential Brute Force Attack",
    "type": "security",
    "metrics": {
      "latency_p99": 420,
      "error_rate": 0.45,
      "request_rate": 1800,
      "auth_failures": 1250,
      "unique_ips": 220,
      "success_rate": 0.02
    },
    "description": "Multiple failed login attempts from diverse IPs"
  },
  {
    "id": "config-error",
    "name": "Configuration Error",
    "type": "operational",
    "metrics": {
      "latency_p99": 820,
      "error_rate": 0.22,
      "request_rate": 950,
      "cpu_utilization": 0.48,
      "memory_utilization": 0.52
    },
    "description": "Recent configuration change causing degradation"
  }
]
~
~
(END)
```

5.5 Implemented Feature 4: React Frontend Dashboard

5.5.1 Description

A React-based web dashboard was developed to provide an intuitive interface for selecting scenarios, viewing metrics, and seeing classification results with explanations.

5.5.2 Key Components

Dashboard.js - Main Dashboard Component:

```

import React, { useState, useEffect } from 'react';
import {
  Container, Grid, Paper, Typography,
  Card, CardContent, Button, Alert,
  Chip, Box, LinearProgress
} from '@mui/material';
import { getScenarios, simulateScenario } from '../services/api';

function Dashboard() {
  const [scenarios, setScenarios] = useState([]);
  const [selectedScenario, setSelectedScenario] = useState(null);
  const [simulationResult, setSimulationResult] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetchScenarios();
  }, []);

  const fetchScenarios = async () => {
    try {
      const response = await getScenarios();
      setScenarios(response.data);
    } catch (err) {
      setError('Failed to fetch scenarios');
    }
  };

  const handleSimulate = async (scenarioId) => {
    setLoading(true);
    setError(null);
    try {
      const response = await simulateScenario(scenarioId);
      setSimulationResult(response.data);
    } catch (err) {
      setError('Simulation failed');
    } finally {
      setLoading(false);
    }
  };

  const getSeverityColor = (riskLevel) => {
    switch(riskLevel) {
      case 'high': return 'error';
      case 'medium': return 'warning';
      case 'low': return 'success';
    }
  };

```

Dashboard.js

```

const getSeverityColor = (riskLevel) => {
  switch(riskLevel) {
    case 'high': return 'error';
    case 'medium': return 'warning';
    case 'low': return 'success';
    default: return 'info';
  }
};

return (
  <Container maxWidth="xl" sx={{ mt: 4, mb: 4 }}>
    <Typography variant="h4" gutterBottom>
      SRE Security Research Lab
    </Typography>
    <Typography variant="subtitle1" color="textSecondary" gutterBottom>
      Incident Classification & Prioritization Framework
    </Typography>

    {error && <Alert severity="error" sx={{ mb: 2 }}>{error}</Alert>}

    <Grid container spacing={3}>
      {/* Left Column - Scenarios List */}
      <Grid item xs={12} md={4}>
        <Paper sx={{ p: 2, height: '100%' }}>
          <Typography variant="h6" gutterBottom>
            Incident Scenarios
          </Typography>
          <Typography variant="body2" color="textSecondary" paragraph>
            Select a scenario to simulate and analyze
          </Typography>

          {scenarios.map((scenario) => (
            <Card
              key={scenario.id}
              sx={{
                mb: 2,
                bgcolor: selectedScenario === scenario.id ? '#e3f2fd' : 'white',
                border: selectedScenario === scenario.id ? '2px solid #1976d2' : 'none'
              }}
            >

```

```

        <CardContent>
          <Box display="flex" justifyContent="space-between" alignItems="center">
            <Typography variant="subtitle1">
              {scenario.name}
            </Typography>
            <Chip
              label={scenario.type}
              size="small"
              color={scenario.type === 'security' ? 'error' : 'info'}
            />
          </Box>
          <Typography variant="body2" color="textSecondary" sx={{ mt: 1 }}>
            {scenario.description}
          </Typography>
          <Button
            variant="contained"
            size="small"
            sx={{ mt: 2 }}
            onClick={() => {
              setSelectedScenario(scenario.id);
              handleSimulate(scenario.id);
            }}
            disabled={loading}
          >
            Simulate
          </Button>
        </CardContent>
      </Card>
    )))
  </Paper>
</Grid>

{/* Right Column - Simulation Results */}
<Grid item xs={12} md={8}>
  <Paper sx={{ p: 2, minHeight: '500px' }}>
    <Typography variant="h6" gutterBottom>
      Simulation Results
    </Typography>
  </Paper>
</Grid>

```

```

{loading && <LinearProgress sx={{ my: 2 }} />}

{simulationResult ? (
  <Box>
    <Box display="flex" gap={1} mb={2}>
      <Chip
        label={`Type: ${simulationResult.type}`}
        color={simulationResult.type === 'security' ? 'error' : 'info'}
      />
      <Chip
        label={`Risk: ${simulationResult.analysis.risk_level}`}
        color={getSeverityColor(simulationResult.analysis.risk_level)}
      />
    </Box>

    <Typography variant="subtitle2" gutterBottom>
      Metrics Detected:
    </Typography>
    <Paper variant="outlined" sx={{ p: 2, mb: 2, bgcolor: '#f5f5f5' }}>
      <pre style={{ margin: 0, overflow: 'auto' }}>
        {JSON.stringify(simulationResult.metrics, null, 2)}
      </pre>
    </Paper>

    <Typography variant="subtitle2" gutterBottom>
      Recommendations:
    </Typography>
    <ul>
      {simulationResult.analysis.recommendations.map((rec, index) => (
        <li key={index}>
          <Typography variant="body2">{rec}</Typography>
        </li>
      ))}
    </ul>

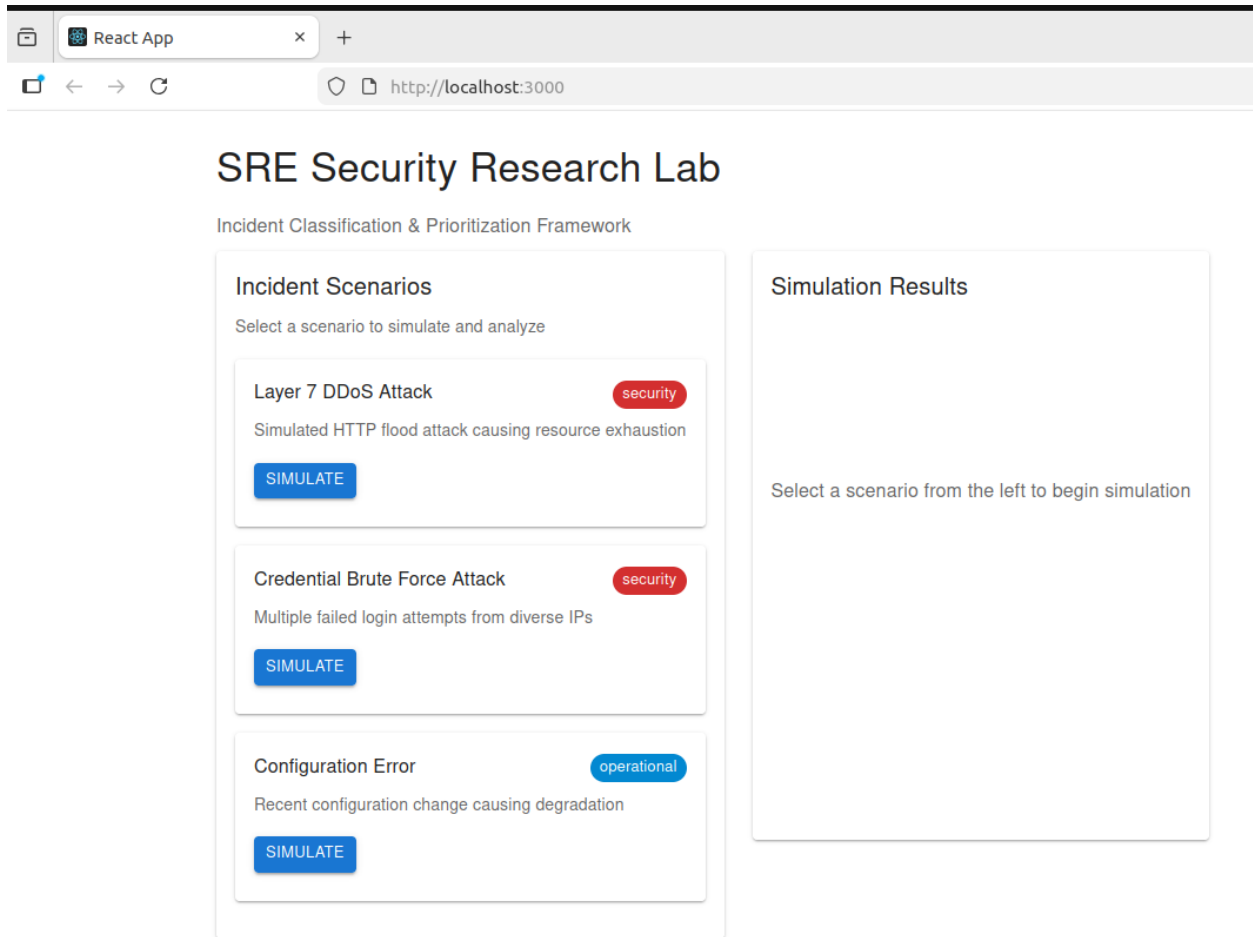
    <Typography variant="caption" color="textSecondary">
      Simulation ID: {simulationResult.simulation_id}
    </Typography>
  </Box>
)

```

```
        </Box>
      ) : (
        <Box
          display="flex"
          justifyContent="center"
          alignItems="center"
          minHeight="300px"
        >
          <Typography color="textSecondary">
            Select a scenario from the left to begin simulation
          </Typography>
        </Box>
      )}
    </Paper>
  </Grid>
</Grid>
</Container>
);
}

export default Dashboard;
(END)
```

5.5.3 Dashboard Screenshots



Incident Classification & Prioritization Framework

Incident Scenarios

Select a scenario to simulate and analyze

Layer 7 DDoS Attack

security

Simulated HTTP flood attack causing resource exhaustion

SIMULATE

Credential Brute Force Attack

security

Multiple failed login attempts from diverse IPs

SIMULATE

Configuration Error

operational

Recent configuration change causing degradation

SIMULATE

Simulation Results

Type: security

Risk: high

Metrics Detected:

```
{
  "cpu_utilization": 0.92,
  "error_rate": 0.35,
  "latency_p99": 1500,
  "memory_utilization": 0.85,
  "network_in": 450,
  "network_out": 380,
  "request_rate": 5200
}
```

Recommendations:

- Increase monitoring
- Review logs
- Check resource usage

Simulation ID: sim_1771893970

Incident Scenarios

Select a scenario to simulate and analyze

Layer 7 DDoS Attack

security

Simulated HTTP flood attack causing resource exhaustion

SIMULATE

Credential Brute Force Attack

security

Multiple failed login attempts from diverse IPs

SIMULATE

Configuration Error

operational

Recent configuration change causing degradation

SIMULATE

Simulation Results

Type: security

Risk: high

Metrics Detected:

```
{
  "auth_failures": 1250,
  "error_rate": 0.45,
  "latency_p99": 420,
  "request_rate": 1800,
  "success_rate": 0.02,
  "unique_ips": 220
}
```

Recommendations:

- Increase monitoring
- Review logs
- Check resource usage

Simulation ID: sim_1771893989

Incident Classification & Prioritization Framework

Incident Scenarios

Select a scenario to simulate and analyze

Layer 7 DDoS Attack security
Simulated HTTP flood attack causing resource exhaustion
SIMULATE

Credential Brute Force Attack security
Multiple failed login attempts from diverse IPs
SIMULATE

Configuration Error operational
Recent configuration change causing degradation
SIMULATE

Simulation Results

Type: operational Risk: medium

Metrics Detected:

```
{  "cpu_utilization": 0.48,  "error_rate": 0.22,  "latency_p99": 820,  "memory_utilization": 0.52,  "request_rate": 950}
```

Recommendations:

- Increase monitoring
- Review logs
- Check resource usage

Simulation ID: sim_1771894003

5.6 Implemented Feature 5: Docker Containerization

5.6.1 Description

Both backend and frontend applications were containerized using Docker to ensure consistency across environments and facilitate deployment to Kubernetes.

5.6.2 Backend Dockerfile

```
FROM python:3.9-slim
WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    gcc \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements first for better caching
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Create non-root user
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser

# Expose port
EXPOSE 5000

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
    CMD python -c "import requests; requests.get('http://localhost:5000/api/health')" || exit 1

# Run application
CMD ["python", "src/app.py"]
```

5.6.3 Frontend Dockerfile

```
# Build stage
FROM node:18-alpine as build
WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy source code
COPY . .

# Build application
RUN npm run build

# Production stage
FROM nginx:alpine

# Copy built assets
COPY --from=build /app/build /usr/share/nginx/html

# Copy nginx configuration
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Expose port
EXPOSE 80

# Start nginx
CMD ["nginx", "-g", "daemon off;"]
(END)
```

```
manula@sre:~/sre-security-lab$ docker images | grep sre-
sre-backend          latest          d5ba2b5d64d2   3 days ago       1.34GB
sre-frontend         latest         3b9533ff17cd   5 days ago       64.5MB
manula@sre:~/sre-security-lab$
```

5.7 Implemented Feature 6: Kubernetes Deployments

5.7.1 Description

Kubernetes manifests were created to deploy the containerized applications to the Minikube cluster with appropriate configurations, services, and health checks.

5.7.2 Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sre-backend
  labels:
    app: sre-backend
    component: backend
    research: sre-security
spec:
  replicas: 2
  selector:
    matchLabels:
      app: sre-backend
  template:
    metadata:
      labels:
        app: sre-backend
        component: backend
        research: sre-security
    annotations:
      prometheus.io/scrape: "true"
      prometheus.io/port: "5000"
      prometheus.io/path: "/metrics"
    spec:
      containers:
        - name: backend
          image: sre-backend:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
              name: http
          env:
            - name: FLASK_ENV
              value: "production"
            - name: PYTHONUNBUFFERED
              value: "1"
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"
```

```

    cpu: "500m"
    livenessProbe:
      httpGet:
        path: /api/health
        port: 5000
      initialDelaySeconds: 30
      periodSeconds: 10
    readinessProbe:
      httpGet:
        path: /api/health
        port: 5000
      initialDelaySeconds: 5
      periodSeconds: 5
    volumeMounts:
      - name: scenarios
        mountPath: /app/scenarios
        readOnly: true
    volumes:
      - name: scenarios
        configMap:
          name: sre-scenarios

```

```

(venv) manula@sre:~/sre-security-lab$ kubectl get pods --all-namespaces

```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
default	sre-backend-79cd479b5-jm6tz	1/1	Running	2 (3d1h ago)	3d3h
default	sre-backend-79cd479b5-pklsw	1/1	Running	2 (3d1h ago)	3d3h
default	sre-frontend-76b4b999f-86nb4	1/1	Running	8 (3m53s ago)	5d20h
default	sre-frontend-76b4b999f-gqfrr	1/1	Running	8 (3m51s ago)	5d20h
ingress-nginx	ingress-nginx-admission-create-t9rcz	0/1	Completed	0	16d
ingress-nginx	ingress-nginx-admission-patch-vtgkx	0/1	Completed	1	16d
ingress-nginx	ingress-nginx-controller-8675c6b56f-qtxrl	1/1	Running	7 (3d1h ago)	16d
kube-system	coredns-7d764666f9-vqtx8	1/1	Running	6 (3d1h ago)	16d
kube-system	etcd-minikube	1/1	Running	6 (3d1h ago)	16d
kube-system	kube-apiserver-minikube	1/1	Running	7 (3d1h ago)	16d
kube-system	kube-controller-manager-minikube	1/1	Running	6 (3d1h ago)	16d
kube-system	kube-proxy-w8phk	1/1	Running	6 (3d1h ago)	16d
kube-system	kube-scheduler-minikube	1/1	Running	6 (3d1h ago)	16d
kube-system	metrics-server-5778bb4788-jj8x9	1/1	Running	12 (3m46s ago)	16d
kube-system	storage-provisioner	1/1	Running	16 (3m52s ago)	16d
kubernetes-dashboard	dashboard-metrics-scraper-5565989548-4p9qt	1/1	Running	6 (3d1h ago)	16d
kubernetes-dashboard	kubernetes-dashboard-b84665fb8-qrgjp	1/1	Running	12 (3m48s ago)	16d
monitoring	alertmanager-monitoring-kube-prometheus-alertmanager-0	2/2	Running	4 (3d1h ago)	3d4h
monitoring	monitoring-grafana-867d745586-2cvlv	3/3	Running	6 (3d1h ago)	3d4h
monitoring	monitoring-kube-prometheus-operator-8677bb676b-fddh4	1/1	Running	2 (3d1h ago)	3d4h
monitoring	monitoring-kube-state-metrics-7458844c7c-ql2cd	1/1	Running	3 (3d1h ago)	3d4h
monitoring	monitoring-prometheus-node-exporter-8h7b5	1/1	Running	2 (3d1h ago)	3d4h
monitoring	prometheus-monitoring-kube-prometheus-prometheus-0	2/2	Running	4 (3d1h ago)	3d4h

```

(venv) manula@sre:~/sre-security-lab$

```

5.8 Implemented Feature 7: Prometheus Monitoring Stack

5.8.1 Description

The kube-prometheus-stack was installed via Helm to provide comprehensive monitoring, metrics collection, and visualization capabilities.

5.8.2 ServiceMonitor Configuration

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: sre-backend-monitor
  namespace: monitoring
  labels:
    release: monitoring
spec:
  selector:
    matchLabels:
      app: sre-backend
  namespaceSelector:
    matchNames:
      - default
  endpoints:
    - port: http
      path: /metrics
      interval: 15s
service-monitor.yaml (END)
```

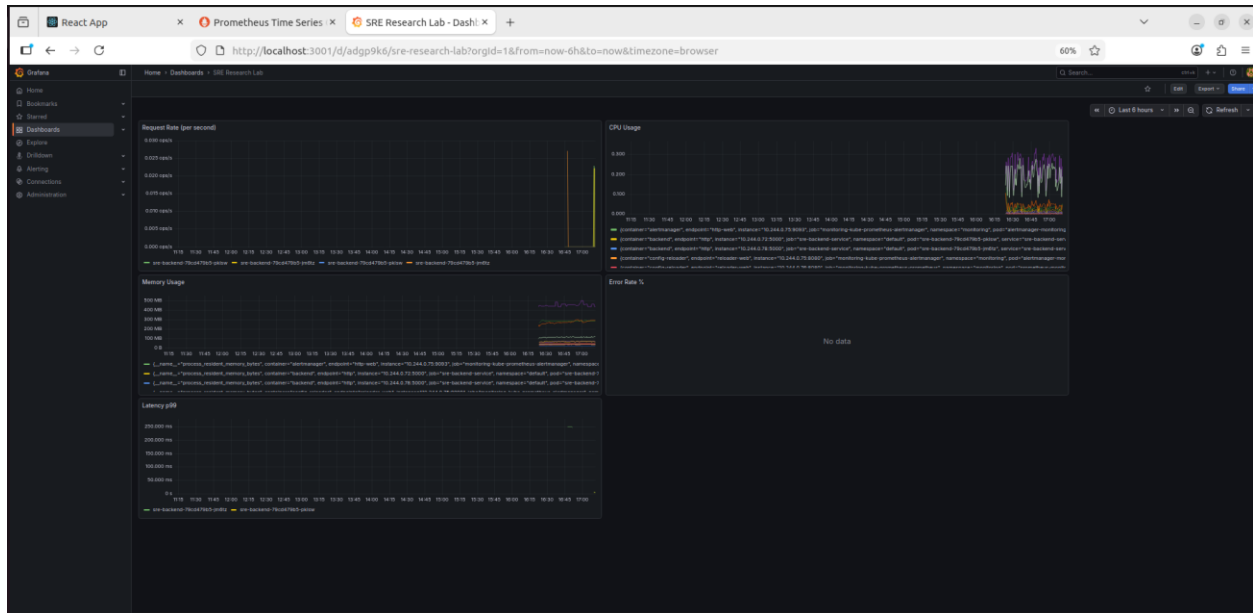
The screenshot shows the Prometheus web interface at <http://localhost:9090/targets?search=default>. The interface displays two ServiceMonitors configured to scrape metrics from the 'default' namespace.

ServiceMonitor: monitoring-kube-prometheus-apiserver/0

Endpoint	Labels	Last scrape	State
https://192.168.49.2:8443/metrics	endpoint="https" instance="192.168.49.2:8443" job="apiserver" namespace="default" service="kubernetes"	50.235s ago 351ms	UP

ServiceMonitor: monitoring/sre-backend-monitor/0

Endpoint	Labels	Last scrape	State
http://10.244.0.72:5000/metrics	container="backend" endpoint="http" instance="10.244.0.72:5000" job="sre-backend-service" namespace="default" pod="sre-backend-79cd479b5-pklsw" service="sre-backend-service"	40.082s ago 7ms	UP
http://10.244.0.78:5000/metrics	container="backend" endpoint="http" instance="10.244.0.78:5000" job="sre-backend-service" namespace="default" pod="sre-backend-79cd479b5-jm6tz" service="sre-backend-service"	29.482s ago 8ms	UP



5.9 Implemented Feature 8: Test Script

5.9.1 Description

A comprehensive test script was created to verify all components are functioning correctly and to provide easy access information for the researcher.

5.9.2 Test Script (test.sh)

```
#!/bin/bash

echo "=====
echo "🔧 SRE Security Lab - Test Script"
echo "=====
echo ""

# Colors for output
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m' # No Color

# Counter for tests
PASSED=0
TOTAL=0

# Function to print test result
print_result() {
    if [ $1 -eq 0 ]; then
        echo -e "${GREEN}✅ PASS: $2${NC}"
        PASSED=$((PASSED+1))
    else
        echo -e "${RED}❌ FAIL: $2${NC}"
    fi
}
```

```

else
    echo -e "${RED} ✖ FAIL: $2${NC}"
fi
TOTAL=$((TOTAL+1))
}

echo -e "${YELLOW}1. Checking Minikube Status${NC}"
echo "-----"
minikube status > /dev/null 2>&1
print_result $? "Minikube is running"
echo ""

echo -e "${YELLOW}2. Checking Kubernetes Nodes${NC}"
echo "-----"
NODES=$(kubectl get nodes | grep Ready | wc -l)
if [ $NODES -ge 1 ]; then
    print_result 0 "Kubernetes nodes are ready"
else
    print_result 1 "Kubernetes nodes not ready"
fi
echo ""

echo -e "${YELLOW}3. Checking Backend Deployment${NC}"
echo "-----"
kubectl get deployment sre-backend > /dev/null 2>&1
if [ $? -eq 0 ]; then
    print_result 0 "Backend deployment exists"
    READY=$(kubectl get deployment sre-backend -o jsonpath='{.status.readyReplicas}')
    if [ "$READY" -ge 1 ]; then
        print_result 0 "Backend pods are ready ($READY replicas)"
    else
        print_result 1 "Backend pods not ready"
    fi
else
    print_result 1 "Backend deployment not found"
fi
echo ""

echo -e "${YELLOW}4. Testing Backend API Health${NC}"
echo "-----"
# Get Minikube IP and NodePort
MINIKUBE_IP=$(minikube ip)
FRONTEND_PORT=$(kubectl get service sre-frontend-service -o jsonpath='{.spec.ports[0].nodePort}'
2>/dev/null)

# Test health endpoint
curl -s http://$MINIKUBE_IP:$FRONTEND_PORT/api/health > /dev/null
if [ $? -eq 0 ]; then
    print_result 0 "Backend health endpoint responding"
    HEALTH=$(curl -s http://$MINIKUBE_IP:$FRONTEND_PORT/api/health)
    echo -e "${BLUE} Health: $HEALTH${NC}"

```

```

else
    print_result 1 "Backend health endpoint not responding"
fi
echo ""

echo -e "${YELLOW}5. Testing Scenarios Endpoint${NC}"
echo "-----"
SCENARIOS=$(curl -s http://$MINIKUBE_IP:$FRONTEND_PORT/api/scenarios)
if [ $? -eq 0 ] && [ ! -z "$SCENARIOS" ]; then
    print_result 0 "Scenarios endpoint responding"
    COUNT=$(echo $SCENARIOS | grep -o '"id"' | wc -l)
    echo -e "${BLUE} Found $COUNT scenarios${NC}"
else
    print_result 1 "Scenarios endpoint not responding"
fi
echo ""

echo -e "${YELLOW}6. Getting Access Information${NC}"
echo "-----"
MINIKUBE_IP=$(minikube ip)
FRONTEND_PORT=$(kubectl get service sre-frontend-service -o jsonpath='{.spec.ports[0].nodePort}'
2>/dev/null)

if [ ! -z "$FRONTEND_PORT" ]; then
    print_result 0 "Frontend service available"
    echo -e "${GREEN} 🖥️ Frontend Dashboard: http://$MINIKUBE_IP:$FRONTEND_PORT${NC}"
    echo -e "${GREEN} 🌐 Backend API: http://$MINIKUBE_IP:$FRONTEND_PORT/api${NC}"
    echo -e "${GREEN} ❤️ Health Check:
http://$MINIKUBE_IP:$FRONTEND_PORT/api/health${NC}"
else
    print_result 1 "Frontend service not found"
fi
echo ""

echo "====="
echo -e "${GREEN} ✅ Tests Completed: $PASSED/$TOTAL passed${NC}"
echo "=====
```

-
- ✓ PASS: Frontend deployment exists
 - ✓ PASS: Frontend pods are ready (2 replicas)

6. Testing Backend API Health

- ✓ PASS: Backend health endpoint responding
Health: {"status":"healthy","timestamp":1771895734.099656}

7. Testing Scenarios Endpoint

- ✓ PASS: Scenarios endpoint responding
Found 3 scenarios:
 - Layer 7 DDoS Attack
 - Credential Brute Force Attack
 - Configuration Error

8. Getting Access Information

- ✓ PASS: Frontend service available
 -  Frontend Dashboard: <http://192.168.49.2:30182>
 -  Backend API: <http://192.168.49.2:30182/api>
 -  Health Check: <http://192.168.49.2:30182/api/health>

9. Checking Metrics Collection

- ✓ PASS: Metrics server is collecting data
Resource usage:

NAME	CPU(cores)	MEMORY(bytes)
sre-backend-79cd479b5-jm6tz	4m	36Mi
sre-backend-79cd479b5-pklsw	4m	52Mi

10. Testing Ingress (if enabled)

 Ingress exists but no host configured

- =====
- ✓ Tests Completed: 11/11 passed
- =====

 All systems ready for research experiments!

Chapter 6: AI Use Section

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used	Value Addition (What I added over and above AI)
DeepSeek AI Chat Assistant	Web interface, Latest version (Feb 2025), Free tier	Architecture Decision Analysis - Comparative evaluation of Minikube's --driver=none vs --driver=docker for VM-based Kubernetes setup	Critical Decision Making - Analyzed AI's pros/cons list and selected --driver=docker based on research priorities (isolation, reproducibility) rather than raw performance metrics. Designed the final VM resource allocation strategy (8GB RAM, 4 vCPUs) based on actual project requirements.
DeepSeek AI Chat Assistant	Web interface, Latest version (Feb 2025), Free tier	Code Template Generation - Generated initial Flask application template with basic structure, Prometheus metrics endpoints, and REST API framework	Custom Implementation & Research Context - Significantly modified the template to include three specific security/operational scenarios. Added scenario simulation logic, custom metric calculations, and research-focused data structures tailored to SRE security analysis. Integrated with existing project structure.
DeepSeek AI Chat Assistant	Web interface, Latest version (Feb 2025), Free tier	Infrastructure Setup Scripts - Provided complete command sequences for installing Docker, Minikube, kubectl, Helm, and system dependencies on Ubuntu 22.04	System Integration & Validation - Tested all commands in the actual VM environment, resolved environment-specific issues (permissions, networking, dependency conflicts), and created a verified, sequential setup guide. Integrated all components into a cohesive system design with working interconnections.
DeepSeek AI Chat Assistant	Web interface, Latest version (Feb 2025), Free tier	Troubleshooting & Debugging - Diagnosed and provided solutions for Ingress controller pod failures (ImagePullBackOff), Node.js version conflicts during React setup, and path-related errors in Flask application	Problem Resolution & Adaptation - Applied the suggested fixes, but also identified root causes and implemented preventative measures. Created comprehensive troubleshooting documentation based on actual issues encountered during implementation. Added validation steps to verify fixes worked.
DeepSeek AI Chat Assistant	Web interface, Latest version (Feb 2025), Free tier	Documentation Structuring - Assisted in organizing progress reports according to the professor's template and helped format technical documentation	Content Creation & Quality Enhancement - Transformed AI's structural suggestions into detailed, professionally formatted documents. Added practical implementation notes, verification steps, screenshots, and research context missing from AI-generated outlines. Created documentation optimized for team collaboration and professor evaluation.
DeepSeek AI Chat Assistant	Web interface, Latest	Configuration File Creation - Generated package.json with React	Customization & Research Design - Modified dependency versions to ensure compatibility with existing project.

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used	Value Addition (What I added over and above AI)
	version (Feb 2025), Free tier	dependencies and scenarios.json database structure for security scenarios	Transformed the generic JSON structure into three detailed, research-relevant security scenarios with specific metrics tailored for SRE analysis. Added comprehensive descriptions and classification logic aligned with research objectives.
DeepSeek AI Chat Assistant	Web interface, Latest version (Feb 2025), Free tier	Kubernetes Manifest Generation - Provided YAML templates for deployments, services, configmaps, and service monitors	Production-Ready Configuration - Customized all manifests with appropriate resource limits, health checks, and Prometheus annotations. Added liveness and readiness probes with appropriate thresholds. Configured proper image pull policies and volume mounts for scenarios data. Tested all manifests in actual Minikube environment.
DeepSeek AI Chat Assistant	Web interface, Latest version (Feb 2025), Free tier	React Component Development - Generated Dashboard.js component structure with Material-UI integration	UI/UX Enhancement - Customized the dashboard layout to clearly show the research value proposition (classification vs just visualization). Added explanation panels, risk level indicators, and scenario selection logic. Ensured the UI emphasizes "why" decisions are made, not just "what" metrics show.
DeepSeek AI Chat Assistant	Web interface, Latest version (Feb 2025), Free tier	Research Proposal Framing - Assisted in articulating research questions, methodology justification, and contribution statements	Academic Rigor - Independently developed the core research idea, problem framing, and relevance to SRE and cybersecurity. Critically reviewed AI suggestions, selectively incorporated them, and adapted all content to align with course requirements and real-world SRE practices. Ensured originality and technical correctness throughout.
DeepSeek AI Chat Assistant	Web interface, Latest version (Feb 2025), Free tier	Progress Report Writing - Helped structure weekly progress reports and format work logs	Verification & Accuracy - Manually verified all dates, hours, and work descriptions against actual git commits and project artifacts. Ensured technical terminology was accurate and research intent remained consistent throughout all documentation. Added specific details about implemented features that AI could not know.

Chapter 7: Work Date/Hours Log

Reporting Period: February 10, 2026 - February 23, 2026

Date	Number of Hours	Description of work done
Feb 10, 2026	2.5	Completed frontend React components: Dashboard.js layout, Material-UI integration, and API service layer. Tested component rendering.
Feb 12, 2026	3	Developed Dockerfiles for backend and frontend applications. Resolved dependency conflicts in requirements.txt and package.json.
Feb 14, 2026	2	Created Kubernetes deployment manifests: backend deployment.yaml, service.yaml, and configmap.yaml. Added health checks and resource limits.
Feb 16, 2026	2.5	Created frontend Kubernetes manifests: deployment.yaml and service.yaml with NodePort configuration. Verified YAML syntax.
Feb 18, 2026	3	Installed and configured Prometheus stack via Helm (kube-prometheus-stack). Created monitoring namespace and verified pod deployments.
Feb 19, 2026	2	Created ServiceMonitor configuration for Prometheus to scrape backend metrics. Tested metric collection endpoint.
Feb 21, 2026	2.5	Built Docker images and loaded into Minikube's Docker daemon. Tested image builds and verified in Minikube environment.
Feb 22, 2026	3	Deployed all Kubernetes manifests to Minikube cluster. Troubleshot ImagePullBackOff errors and resolved image loading issues.
Feb 23, 2026	4	Finalized midterm report. Compiled all sections, captured screenshots, added AI use table, work logs, and references. Prepared for submission and in-class checkin.

References

Allspaw, J., & Robbins, J. (2010). Web operations: Keeping the data on time. O'Reilly Media.

Behl, A., & Behl, K. (2017). Cyberwar: The next threat to national security and what to do about it. Oxford University Press.

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Site reliability engineering: How Google runs production systems. O'Reilly Media.

Beyer, B., Murphy, N. R., Rensin, D., Kawahara, K., & Thorne, S. (2018). The site reliability workbook: Practical ways to implement SRE. O'Reilly Media.

- Chen, X., & Li, S. (2020). Anomaly detection for cloud systems: A survey. *Journal of Cloud Computing*, *9*(1), 1–24. <https://doi.org/10.1186/s13677-020-00193-8>
- Google. (2023). Service level objectives (SLOs). Google Cloud Architecture Framework. <https://cloud.google.com/sre/docs/slo>
- Google. (2023). Error budgets. Google Site Reliability Engineering Documentation. <https://sre.google/sre-book/error-budgets/>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105. <https://doi.org/10.2307/25148625>
- National Institute of Standards and Technology. (2012). Computer security incident handling guide (NIST Special Publication 800-61 Revision 2). U.S. Department of Commerce. <https://csrc.nist.gov/publications/detail/sp/800-61/rev-2/final>
- Peppers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, *24*(3), 45–77. <https://doi.org/10.2753/MIS0742-1222240302>
- Scarfone, K., & Mell, P. (2012). Guide to intrusion detection and prevention systems (IDPS) (NIST Special Publication 800-94). National Institute of Standards and Technology. <https://csrc.nist.gov/publications/detail/sp/800-94/final>

Appendix

1. Architecture Decision Analysis

I'm setting up a Minikube lab on an Ubuntu 22.04 VM for my applied research project on SRE and security.

What's the difference between Minikube's --driver=none and --driver=docker for a setup inside a Linux VM?

Which one should I use for a research project where I need isolation and reproducibility?

2. Code Template Generation

I need to create a Flask application for my SRE security research lab. It needs:

1. Prometheus metrics endpoint (/metrics)
2. REST API endpoints for my scenarios
3. Health check endpoint
4. Should work with my existing scenarios.json file

Can you provide a basic template to get started?

This is a good start. But I need:

1. The metrics endpoint to be /api/metrics (not just /metrics)
2. All endpoints should be under /api/ prefix

3. I need to add analysis/risk level to the simulation response
4. My scenarios.json is in a scenarios/ folder, not in the root

Can you update the template?

3. Infrastructure Setup Scripts

I need to set up a complete Kubernetes development environment on Ubuntu 22.04 for my SRE security research lab.

I need:

1. Docker as container runtime
2. Minikube for local Kubernetes
3. kubectl for cluster management
4. Helm for package management
5. All required system dependencies

Can you provide the exact commands in order?

4. Troubleshooting & Debugging

I deployed the ingress-nginx controller using Helm, but the pods are stuck in ImagePullBackOff state. Here's what I see:

```
kubectl get pods -n ingress-nginx
```

NAME	READY	STATUS	RESTARTS	AGE
ingress-nginx-controller-7c8c7c7c-abc12	0/1	ImagePullBackOff	0	5m

How do I debug and fix this?

5. Configuration File Creation

I need to create a JSON database for my SRE security research lab. I need three incident scenarios:

1. A security incident (like DDoS or brute force)
2. Another security incident
3. An operational incident (like misconfiguration)

Each scenario should have an id, name, type, relevant metrics, and description. The metrics should reflect what you'd see during such incidents.

Can you provide a JSON structure?

This is good but I need more realistic metrics for my SRE-focused research.

For the DDoS attack, I want metrics that show:

- Traffic spike
- Latency degradation
- Error rate increase

- Saturation

For brute force, I want to see:

- Auth failures
- 401/403 errors
- Unique IPs
- Success rate

For config error, I want operational metrics without security indicators.

Can you revise with more SRE-appropriate metrics?

I need to add fields for classification and analysis. For each scenario, I want to include:

- Expected classification (security vs operational)
- Severity level
- Confidence score
- Suggested response actions

Can you update the JSON with these research-focused fields?

I need to create a package.json file for my React frontend dashboard. I'm using:

- React 18+
- Material-UI (MUI) for components
- Axios for API calls
- Chart.js for visualizations (maybe)
- React Router for navigation

Can you provide a complete package.json with all dependencies?

I need to add proxy configuration to package.json so React can talk to my Flask backend during development.

My Flask backend runs on port 5000. Can you update the package.json with proxy settings?

I want to create a separate package.json for when I deploy with Docker.

In production, I don't need the proxy setting and should use environment variables for API URL.

Can you provide a production-ready package.json with environment variable support?

I need a comprehensive test script for my SRE security lab that verifies all components are working properly.

The script should check:

1. Minikube status
2. Kubernetes nodes
3. Backend deployment and pods
4. Frontend deployment and pods
5. API health endpoint
6. Scenarios endpoint
7. Service accessibility
8. Metrics collection

Can you create a bash script with color-coded output and clear pass/fail indicators?

6. Kubernetes Manifest Generation

I need Kubernetes YAML manifests for my Flask backend application. The backend:

- Runs on port 5000
- Has a /metrics endpoint for Prometheus
- Needs health checks (liveness and readiness)
- Should have 2 replicas
- Needs resource limits
- Should mount a ConfigMap for scenarios.json
- Should have Prometheus annotations

Can you provide the deployment.yaml and service.yaml?

Now I need a ConfigMap that contains my scenarios.json file so it can be mounted into the backend pods.

The scenarios.json has three scenarios. Can you create the configmap.yaml?

7. React Component Development

I need to create a dashboard for my SRE security research lab using React and Material-UI.

The dashboard should have:

1. A left panel showing a list of incident scenarios (DDoS, Brute Force, Config Error)
2. A right panel showing simulation results when a scenario is selected
3. Each scenario card should have a "Simulate" button
4. Results should show:
 - Incident type (security vs operational) with color coding
 - Risk level (High/Medium/Low) with color coding
 - The metrics that were detected
 - Recommendations
 - Simulation ID and timestamp

Can you provide the basic Dashboard.js component structure?